Dicionário de Dicionários em Python

Sumário

1. Apresentação
2. O que você vai aprender
3. Módulo 1: Revisão - O que é um dicionário?
4. Módulo 2: O Problema - Juntando informações
5. Módulo 3: A Solução - Dicionário de Dicionários
6. Módulo 4: Operações - Trabalhando com a estrutura
7. Módulo 5: Aprofundamento - Indo além
8. Resumo Final e Desafio

---

1. Apresentação

Vamos aprender um dos conceitos mais importantes em Python: **dicionário de dicionários**.

Nossa jornada será assim:

- Começaremos com uma situação simples
- Veremos uma primeira solução
- Descobriremos os problemas dessa solução
- Aprenderemos uma maneira muito melhor
- Praticaremos com muitos exemplos

---

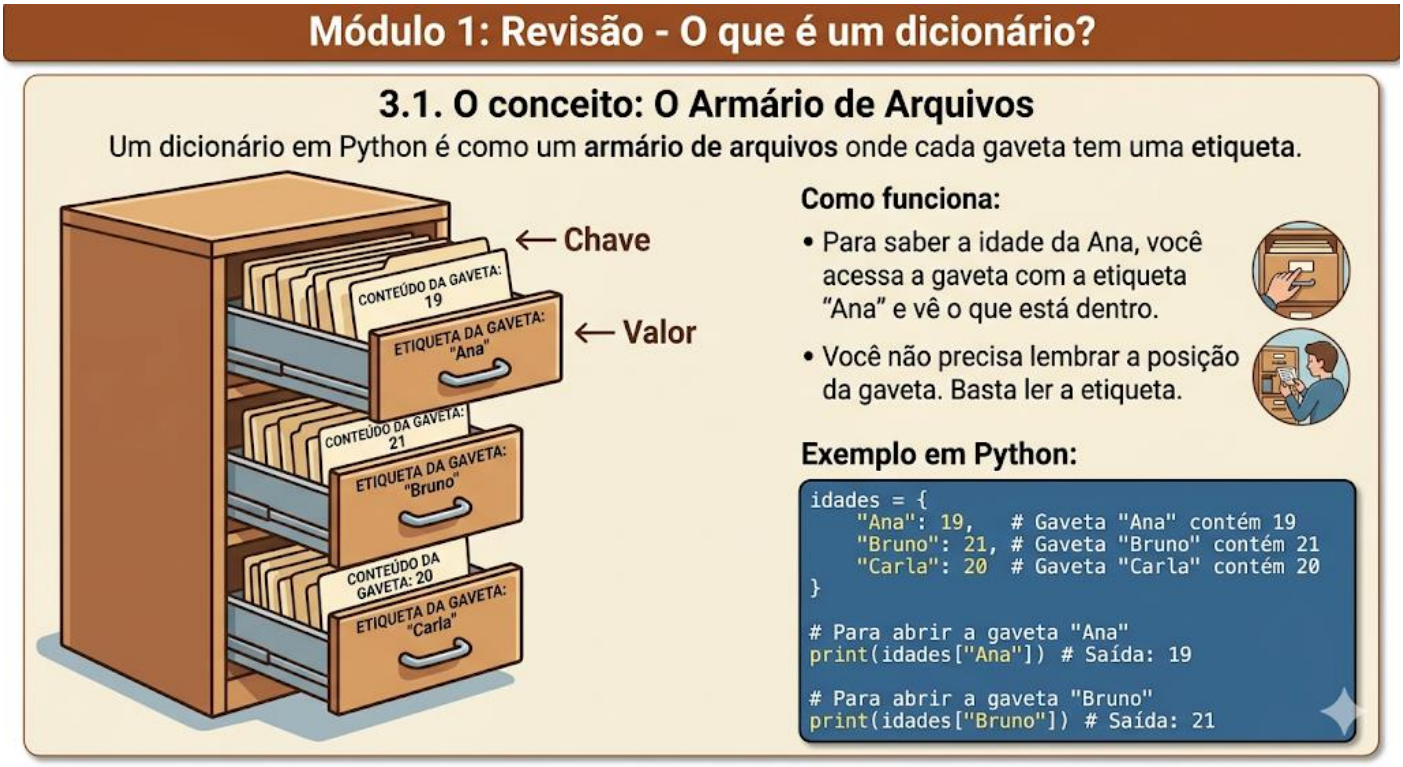
2. O que você vai aprender

Ao final deste material, você será capaz de:

- ✅ Entender o que é um dicionário em Python
- ✅ Juntar informações de vários dicionários em um só
- ✅ Identificar quando uma estrutura está confusa
- ✅ Construir dicionários de dicionários com campos nomeados
- ✅ Consultar, alterar e percorrer essas estruturas
- ✅ Filtrar e agrupar informações
- ✅ Usar os métodos `get()` e `setdefault()` para escrever código mais limpo

Módulo 1: Revisão - O que é um dicionário?

Um dicionário em Python é como um **armário de arquivos** onde cada gaveta tem uma **etiqueta**.



1.1. O conceito

Um dicionário em Python guarda dados em pares: **chave** → **valor**.

Exemplo em Python:

```
idades = {
    "Ana": 19,      # Chave "Ana" → valor 19
    "Bruno": 21,    # Chave "Bruno" → valor 21
    "Carla": 20     # Chave "Carla" → valor 20
}

# Para acessar o valor da chave "Ana"
print(idades["Ana"])   # Saída: 19

# Para acessar o valor da chave "Bruno"
print(idades["Bruno"]) # Saída: 21
```

1.2. Criando um dicionário vazio e adicionando itens

```
# Dicionário vazio
meu_dict = {}
```

```python
# Adicionando um par: chave "João" com valor 25
meu_dict["João"] = 25

# Adicionando mais pares
meu_dict["Maria"] = 30
meu_dict["Pedro"] = 22

print(meu_dict)
# Saída: {'João': 25, 'Maria': 30, 'Pedro': 22}
```

1.3. Visualizando como uma tabela

Chave	Valor
"Ana"	19
"Bruno"	21
"Carla"	20

1.4. Exercício guiado 1
Crie um dicionário chamado "notas" com 3 alunos e suas notas:
python
```python
# Exemplo:
precos = {
    "Arroz": 22.90,
    "Feijão": 8.50,
    "Macarrão": 5.75
}

# Agora é sua vez! Crie um dicionário chamado "notas"
# com 3 alunos (chaves) e suas notas (valores)
```

Gabarito

```python
notas = {
    "João": 8.5,
    "Maria": 9.0,
    "Pedro": 7.5
}
```

Agora, se você quiser saber a nota da Maria, é só usar a chave:
```python
print(notas["Maria"])  # Saída: 9.0
```

Mini Teste 1
1. Em um dicionário em Python, o que identifica cada valor?
a) o índice numérico
b) a chave
c) a posição na memória
d) a linha do programa

2. Qual a sintaxe correta para acessar o valor da chave "Ana" no dicionário `idades`?
a) `idades(Ana)`
b) `idades["Ana"]`
c) `idades[Ana]`
d) `idades.Ana`

3. Como você cria um dicionário vazio?

a) `dict = []`
b) `dict = ()`
c) `dict = {}`
d) `dict = empty()`

Gabarito

1. **b)** a chave
2. **b)** `idades["Ana"]`
3. **c)** `dict = {}`

Módulo 2: O Problema - Juntando informações

4. Situação real: dois dicionários separados

Vamos imaginar um sistema de uma escola. Temos dois dicionários separados:

Dicionário 1: Matrícula → Login

```python
matr_login = {
    "202301": "ana.silva",
    "202302": "bruno.souza",
    "202303": "carla.melo"
}
```

Dicionário 2: Login → Senha

```python
login_senha = {
    "ana.silva": "1234",
    "bruno.souza": "abcd",
    "carla.melo": "xyz"
}
```

O problema: Queremos um **único dicionário** onde, a partir da matrícula, possamos encontrar **login e senha juntos**.

5. Primeira tentativa: usando tupla

Vamos criar um novo dicionário. Para cada matrícula, vamos colocar como valor uma **tupla** contendo login e senha.

```
usuarios = {}  # Começamos com um dicionário vazio

# Para cada aluno  no primeiro dicionário, Passo 1: Abrir a gaveta da matrícula no Armário 1 e
pegar o login

for (matr,login) in matr_login.items():

    # Passo 2: Usar o login para abrir a gaveta no Armário 2 e pegar a senha
    senha = login_senha.get(login)

    # Passo 3:caso o aluno tenha senha,
              # colocar no novo armário uma gaveta com etiqueta = matrícula
              # Dentro da gaveta, colocamos um envelope com (login, senha)
    if senha:
        usuarios[matr] = (login, senha)
```

Resultado:

```
{
    "202301": ("ana.silva", "1234"),
    "202302": ("bruno.souza", "abcd"),
    "202303": ("carla.melo", "xyz")
}
```

Iteração	`matr`	`login`	`senha`	Estado do dicionário `usuarios`
1	`"202301"`	`"ana.silva"`	`"1234"`	`{"202301": ("ana.silva", "1234")}`
2	`"202302"`	`"bruno.souza"`	`"abcd"`	`{"202301": ..., "202302": ("bruno.souza", "abcd")}`
3	`"202303"`	`"carla.melo"`	`"xyz"`	`{"202301": ..., "202302": ..., "202303": ("carla.melo", "xyz")}`

5.2. Exercício guiado 2
Agora é sua vez!

```
matr_login = {
    "202401": "luiza.rosa",
    "202402": "marcos.lima",
    "202403": "nina.alves"
}

login_senha = {
    "luiza.rosa": "aa11",
    "marcos.lima": "bb22",
    "nina.alves": "cc33"
}
```

Tarefa 1: Explique, com palavras, o caminho usado para descobrir a senha da matrícula "202402".

`Gabarito`
Explicação para "202402":

1. Procura-se a chave "202402" no dicionário `matr_login`
2. Encontra-se o valor "marcos.lima" (login)
3. Procura-se a chave "marcos.lima" no dicionário `login_senha`
4. Encontra-se o valor "bb22" (senha)

---

6. O problema da tupla

Agora temos um dicionário funcionando. Mas será que ele é fácil de usar?

Vamos acessar os dados:

```
usuarios["202301"][0]  # O que está na posição 0 da tupla?
usuarios["202301"][1]  # O que está na posição 1 da tupla?
```

Pergunta: Você consegue dizer, sem olhar, o que significa `[0]`? E `[1]`?

Problema 1: Precisamos decorar as posições

- Posição 0 = login
- Posição 1 = senha

Isso funciona para 2 informações, mas e se tivermos 3, 4 ou 5?

Problema 2: O código não explica o que faz

Compare:

```
# O que isso significa? É o login? É a senha?
usuarios["202301"][0]

# Agora compare com:
usuarios["202301"]["login"]
```

A segunda forma **diz o que está fazendo**: "login do usuário 202301"

Problema 3: Dificuldade para expandir

E se quisermos adicionar mais informações (como o curso do aluno)? Com a tupla:

```
# Antes: ("ana.silva", "1234")
# Depois: ("ana.silva", "1234", "Informática")
```

Agora temos que lembrar:

- Posição 0 = login
- Posição 1 = senha
- Posição 2 = curso

A dificuldade aumenta rapidamente!

6.1. Teste rápido
Tente acessar a senha do aluno 202303 usando a posição da tupla. Agora responda: era posição 0 ou 1?

```
# Tente sem olhar
print(usuarios["202303"][?])  # Qual número?
```

Gabarito
A posição é **1** (senha está na segunda posição, índice 1).

```
print(usuarios["202303"][1])  # Saída: xyz
```

Conseguiu lembrar sem olhar? Se sim, ótimo! Mas e daqui a uma semana? É exatamente esse o problema!

Mini Teste 2
1. No valor `("Informática", "Manhã")`, a informação está organizada como:
a) um dicionário
b) uma lista
c) uma tupla
d) um conjunto
2. Qual é a principal limitação da tupla neste contexto?
a) não guarda texto
b) obriga a decorar a ordem das informações
c) não permite ser usada como valor
d) apaga as chaves
3. O que acontece se adicionarmos mais informações à tupla?
a) fica mais fácil de usar
b) não muda nada
c) fica mais difícil lembrar o que cada posição significa
d) o Python não permite

Gabarito
1. **c)** uma tupla
2. **b)** obriga a decorar a ordem das informações
3. **c)** fica mais difícil lembrar o que cada posição significa

---

Módulo 3: A Solução - Dicionário de Dicionários

7. A estrutura ideal

A solução é usar **dicionário de dicionários** - ou seja, cada valor do dicionário principal é **outro dicionário** com campos nomeados.

Nossa estrutura melhorada:

```
usuarios = {
    "202301": {"login": "ana.silva", "senha": "1234"},
    "202302": {"login": "bruno.souza", "senha": "abcd"},
    "202303": {"login": "carla.melo", "senha": "xyz"}
}
```

Como acessar:

```
# Acessando o sub-dicionário do usuário
dados_usuario = usuarios.get("202301", {})

# Acessando campos dentro do sub-dicionário
login = dados_usuario.get("login", "Não informado")
senha = dados_usuario.get("senha", "Não informado")

print(login)   # ana.silva
print(senha)   # 1234

# Em uma única linha (encadeamento de get)
print(usuarios.get("202301", {}).get("login", "Não informado"))  # ana.silva
```

Leia:

→"No dicionário usuarios, tente acessar a chave 202301. Se existir, acesse o login. Se não existir, retorne 'Não informado'"

7.1. Visualizando a estrutura

```
DICIONÁRIO EXTERNO (PRINCIPAL - usuarios)
│
├── CHAVE "202301" → DICIONÁRIO INTERNO
│   ├── "login" → "ana.silva"
│   └── "senha" → "1234"
│
├── CHAVE "202302" → DICIONÁRIO INTERNO
│   ├── "login" → "bruno.souza"
│   └── "senha" → "abcd"
│
└── CHAVE "202303" → DICIONÁRIO INTERNO
    ├── "login" → "carla.melo"
    └── "senha" → "xyz"
```

---

8. Transformando nosso código

```python
usuarios = {}  # Dicionário principal vazio

for (matr,login) in matr_login.items():
    senha = login_senha.get(login,'')
    if senha:
        # Agora, o valor é um SUB-DICIONÁRIO (dicionário interno)
        usuarios[matr] = {
                "login": login,
                "senha": senha
        }
```

Resultado:

```
{
    "202301": {"login": "ana.silva", "senha": "1234"},
    "202302": {"login": "bruno.souza", "senha": "abcd"},
    "202303": {"login": "carla.melo", "senha": "xyz"}
}
```

---

9. Comparação: Tupla vs Dicionário Interno

ANTES (Tupla)

`("ana.silva", "1234")`

Acesso: `[0]` e `[1]`

Precisa decorar a ordem

Difícil de expandir

DEPOIS (Dicionário Interno)

`{"login": "ana.silva", "senha": "1234"}`

Acesso: `["login"]` e `["senha"]`

As chaves mostram o significado

Fácil de expandir

---

10. Outra possibilidade: usar o login como chave interna

Dependendo da necessidade, podemos organizar os dados de forma diferente. Em vez de usar chaves como "login" e "senha", podemos usar o próprio **login como chave interna** e a senha como valor.

Estrutura alternativa:

```
usuarios = {
    "202301": {"ana.silva": "1234"},
    "202302": {"bruno.souza": "abcd"},
    "202303": {"carla.melo": "xyz"}
}
```

Interpretação:

- Chave externa → matrícula
- Chave interna → login
- Valor interno → senha

Código para construir esta estrutura:

```
usuarios = {}  # Dicionário principal vazio

for (matr,login) in matr_login.items():
    senha = login_senha.get(login)
    if senha:
        # O valor agora é um dicionário onde a chave é o login e o valor é a senha
        usuarios[matr] = {login: senha}


print(usuarios)
```

Saída:

```
{
    "202301": {"ana.silva": "1234"},
    "202302": {"bruno.souza": "abcd"},
    "202303": {"carla.melo": "xyz"}
}
```

Acesso nesse modelo:

```
# Acessar a senha do aluno 202301
print(usuarios["202301"]["ana.silva"])  # Saída: 1234

# Mas como saber qual é o login para acessar?
# Para isso, precisaríamos saber o login previamente ou usar o get e tratar possíveis erros
```

Comparação entre as duas formas:

Estrutura	Exemplo	Quando usar
Campos nomeados	`{"login": "ana.silva", "senha": "1234"}`	Quando você precisa armazenar **vários atributos** e quer que cada um tenha um **nome claro**
Login como chave	`{"ana.silva": "1234"}`	Quando você só precisa armazenar **uma informação** relacionada ao login (como a senha) e **já conhece o login** no momento do acesso

💡 **IDEIA-CHAVE:**
Para a maioria das aplicações que exigem múltiplos atributos, a estrutura com **campos nomeados** é preferível porque:

- É mais clara
- É mais fácil de expandir
- Não exige conhecimento prévio de valores internos para acessar os dados

```
# Recomendado para a maioria dos casos
usuarios = {
    "202301": {"login": "ana.silva", "senha": "1234", "curso": "Informática"}
}

# Alternativa (mais limitada)
usuarios = {
    "202301": {"ana.silva": "1234"}
}
```

11. Expandindo para 3 informações

E se quisermos adicionar o curso do aluno?

Com tupla (piora):

```
("ana.silva", "1234", "Informática")
# Agora temos que lembrar: posição 0 = login, 1 = senha, 2 = curso
```

Com dicionário interno:

```
{
    "login": "ana.silva",
    "senha": "1234",
    "curso": "Informática"
}
```

Agora fica claro: cada informação tem sua própria chave!

---

12. Exemplo com 3 dicionários de origem

```
# Dicionários de origem
matr_login = {
    "202301": "ana.silva",
    "202302": "bruno.souza",
    "202303": "carla.melo"
}
```

```python
login_senha = {
    "ana.silva": "1234",
    "bruno.souza": "abcd",
    "carla.melo": "xyz"
}

cursos = {
    "202301": "Informática",
    "202302": "Administração"
    # "202303" não tem curso cadastrado!
}

usuarios = {}

# Usando items() para percorrer o dicionário principal
for (matr, login) in matr_login.items():
    # Usando get() para acessar senha e curso
    senha = login_senha.get(login, "")
    curso = cursos.get(matr, "Não informado")

    usuarios[matr] = {
        "login": login,
        "senha": senha,
        "curso": curso
    }

print(usuarios)
```

Saída:

```
{
    '202301': {'login': 'ana.silva', 'senha': '1234', 'curso': 'Informática'},
    '202302': {'login': 'bruno.souza', 'senha': 'abcd', 'curso': 'Administração'},
    '202303': {'login': 'carla.melo', 'senha': 'xyz', 'curso': 'Não informado'}
}
```

11.1. Visualizando a estrutura final

```
DICIONÁRIO EXTERNO (principal cadastro)
│
├── CHAVE "A101"  - DICIONÁRIO INTERNO
│   ├── "curso" → "Informática"
│   ├── "turno" → "Manhã"
│   └── "media" → 8.5
│
├── CHAVE "A102" → DICIONÁRIO INTERNO
│   ├── "curso" → "Administração"
│   ├── "turno" → "Noite"
│   └── "media" → 7.0
│
└── CHAVE "A103" → DICIONÁRIO INTERNO
    ├── "curso" → "Enfermagem"
    ├── "turno" → "Tarde"
    └── "media" → 9.2
```

Mini Teste 3
Considere:

```python
cadastro = {
```

```
    "A101": {"curso": "Informática", "turno": "Manhã", "media": 8.5}
}
```

1. O que retorna `cadastro["A101"]`?
a) "Informática"
b) o sub-dicionário do aluno A101: `{"curso": "Informática", "turno": "Manhã", "media": 8.5}`
c) 8.5
d) "A101"

2. O que retorna `cadastro["A101"]["curso"]`?
a) "Informática"
b) "Manhã"
c) 8.5
d) `{"curso": "Informática"}`

3. O que retorna `cadastro["A101"]["media"]`?
a) "Informática"
b) "Manhã"
c) 8.5
d) `{"media": 8.5}`

Gabarito

1. **b)** o sub-dicionário do aluno A101
2. **a)** "Informática"
3. **c)** 8.5

---

13. Outro exemplo: Produtos

Vamos aplicar o mesmo raciocínio em um contexto diferente: um sistema de estoque.

Dicionários de origem:

```
# Dicionário de preços
precos = {
    "P10": 12.5,
    "P20": 8.0,
    "P30": 15.9,
    "P40": 25.0
}

# Dicionário de estoques (alguns produtos podem não ter estoque)
estoques = {
    "P10": 20,
    "P20": 35,
    "P30": 10
    # P40 não tem estoque cadastrado!
}

# Dicionário de categorias
categorias = {
    "P10": "Limpeza",
    "P20": "Alimento",
    "P30": "Higiene",
    "P40": "Eletrônicos"
}
```

Construindo o dicionário de dicionários com `items()` e `get()`:

```
produtos = {}

for codigo, preco in precos.items():
    produtos[codigo] = {
        "preco": preco,                                  # veio do items()
        "estoque": estoques.get(codigo, 0),              # se não tem, 0
        "categoria": categorias.get(codigo, "Não categorizado")
    }

print(produtos)
```

Saída:

```
{
    'P10': {'preco': 12.5, 'estoque': 20, 'categoria': 'Limpeza'},
    'P20': {'preco': 8.0, 'estoque': 35, 'categoria': 'Alimento'},
    'P30': {'preco': 15.9, 'estoque': 10, 'categoria': 'Higiene'},
    'P40': {'preco': 25.0, 'estoque': 0, 'categoria': 'Eletrônicos'}
}
```

Observação: O produto P40 recebeu `estoque: 0` porque não tinha estoque cadastrado. O programa não gerou erro!

13.1. Consultando dados com `get()`

```
# Acessando informações de um produto específico
codigo = "P20"
produto = produtos.get(codigo, {})

print(f"Produto {codigo}:")
print(f"  Preço: R$ {produto.get('preco', 0.0)}")
print(f"  Estoque: {produto.get('estoque', 0)} unidades")
print(f"  Categoria: {produto.get('categoria', '?')}")

# Acessando diretamente (encadeamento de get)
print(f"\nPreço do P30: R$ {produtos.get('P30', {}).get('preco', 0.0)}")
print(f"Estoque do P40: {produtos.get('P40', {}).get('estoque', 0)} unidades")
```

Saída:

```
Produto P20:
  Preço: R$ 8.0
  Estoque: 35 unidades
  Categoria: Alimento

Preço do P30: R$ 15.9
Estoque do P40: 0 unidades
```

13.2. Operações com produtos usando `items()`

Listando todos os produtos:

python
```python
print("=" * 50)
print("LISTA COMPLETA DE PRODUTOS")
print("=" * 50)

for codigo, dados in produtos.items():
    print(f"\nCódigo: {codigo}")
    print(f"  Preço: R$ {dados.get('preco', 0.0):.2f}")
    print(f"  Estoque: {dados.get('estoque', 0)} unidades")
    print(f"  Categoria: {dados.get('categoria', '?')}")
```

Filtrando produtos com estoque baixo:

```python
print("\n--- Produtos com estoque baixo (< 15 unidades) ---")
for codigo, dados in produtos.items():
    estoque = dados.get('estoque', 0)
    if estoque < 15:
        print(f"  {codigo}: {estoque} unidades")
```

Saída:

```
--- Produtos com estoque baixo (< 15 unidades) ---
  P30: 10 unidades
  P40: 0 unidades
```

13.3. Agrupamento de produtos por categoria

```python
# Agrupando produtos por categoria usando setdefault()
produtos_por_categoria = {}

for codigo, dados in produtos.items():
    categoria = dados.get('categoria', 'Sem categoria')
    produtos_por_categoria.setdefault(categoria, []).append(codigo)

print("\n--- Produtos agrupados por categoria ---")
for categoria, lista in produtos_por_categoria.items():
    print(f"{categoria}: {lista}")
```

Saída:

```
--- Produtos agrupados por categoria ---
Limpeza: ['P10']
Alimento: ['P20']
Higiene: ['P30']
Eletrônicos: ['P40']
```

13.4. Valor total em estoque por categoria

```python
# Calculando o valor total em estoque por categoria
valor_por_categoria = {}

for codigo, dados in produtos.items():
    categoria = dados.get('categoria', 'Sem categoria')
    preco = dados.get('preco', 0.0)
    estoque = dados.get('estoque', 0)
    valor_total = preco * estoque
```

```python
    valor_por_categoria[categoria] = valor_por_categoria.get(categoria, 0.0) + valor_total

print("\n--- Valor total em estoque por categoria ---")
for categoria, valor in valor_por_categoria.items():
    print(f"{categoria}: R$ {valor:.2f}")
```

Saída:

```
--- Valor total em estoque por categoria ---
Limpeza: R$ 250.00
Alimento: R$ 280.00
Higiene: R$ 159.00
Eletrônicos: R$ 0.00
```

13.5. Exercício guiado: Produtos

python
```python
# Dicionários de origem
precos = {
    "TV01": 1999.90,
    "CEL01": 1499.90,
    "NOTE01": 2499.90,
    "FONE01": 99.90
}

estoques = {
    "TV01": 5,
    "CEL01": 12,
    "NOTE01": 3
    # FONE01 não tem estoque
}

categorias = {
    "TV01": "Eletrônicos",
    "CEL01": "Eletrônicos",
    "NOTE01": "Informática",
    "FONE01": "Acessórios"
}

# Tarefa 1: Crie o dicionário "produtos" usando items() e get()
# Tarefa 2: Mostre todos os produtos com estoque menor que 5
# Tarefa 3: Calcule o valor total do estoque
```

Gabarito

```python
# Tarefa 1
produtos = {}
for codigo, preco in precos.items():
    produtos[codigo] = {
        "preco": preco,
        "estoque": estoques.get(codigo, 0),
        "categoria": categorias.get(codigo, "Não categorizado")
    }

# Tarefa 2
print("Produtos com estoque baixo (< 5):")
for codigo, dados in produtos.items():
    if dados.get('estoque', 0) < 5:
        print(f"  {codigo}: {dados.get('estoque', 0)} unidades")

# Tarefa 3
```

```
valor_total = 0
for codigo, dados in produtos.items():
    valor_total += dados.get('preco', 0.0) * dados.get('estoque', 0)
print(f"\nValor total do estoque: R$ {valor_total:.2f}")
```

</details>

---

14. O padrão que se repete

Perceba que a lógica é sempre a mesma:

```
novo_dicionario = {}

# Escolha um dicionário como principal e use items()
for chave, valor_principal in dicionario_principal.items():
    novo_dicionario[chave] = {
        "campo_principal": valor_principal,              # veio do items()
        "campo_secundario1": dicionario2.get(chave, "padrão1"),
        "campo_secundario2": dicionario3.get(chave, "padrão2"),
        # ... mais campos conforme necessário
    }
```

PASSO MENTAL:

Sempre que houver vários dicionários com as mesmas chaves, crie um novo dicionário. Para cada chave, coloque como valor um dicionário interno com campos nomeados para cada informação!

14.1. Exemplos do padrão em diferentes contextos

Exemplo 1: Alunos

```
for codigo, curso in cursos.items():
    cadastro[codigo] = {
        "curso": curso,
        "turno": turnos.get(codigo, "Não informado"),
        "media": medias.get(codigo, 0.0)
    }
```

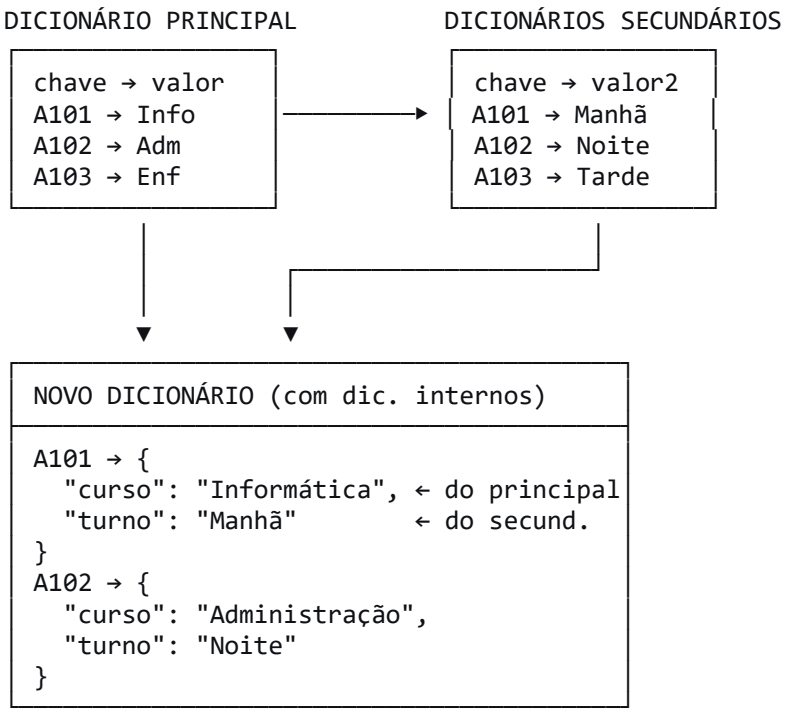
Exemplo 2: Produtos

```
for codigo, preco in precos.items():
    produtos[codigo] = {
        "preco": preco,
        "estoque": estoques.get(codigo, 0),
        "categoria": categorias.get(codigo, "Não categorizado")
    }
```

Exemplo 3: Usuários

```
for matr, login in matr_login.items():
    usuarios[matr] = {
        "login": login,
        "senha": login_senha.get(login, "Não informada")
    }
```

14.2. Diagrama visual do padrão



14.3. Exercício: Identifique o padrão

Considere os dicionários abaixo:

```
alunos_nomes = {
    "001": "Ana Souza",
    "002": "Bruno Lima",
    "003": "Carla Mendes"
}

alunos_idades = {
    "001": 19,
    "002": 21
    # 003 não tem idade
}

alunos_turmas = {
    "001": "A",
    "002": "B",
    "003": "C"
}
```

Escreva o código para criar um dicionário `alunos` usando o padrão com `items()` e `get()`.

```
Gabarito
alunos = {}

for codigo, nome in alunos_nomes.items():
    alunos[codigo] = {
        "nome": nome,
        "idade": alunos_idades.get(codigo, "Não informada"),
        "turma": alunos_turmas.get(codigo, "Não informada")
    }

print(alunos)
```

Mini Teste 4
Marque V (Verdadeiro) ou F (Falso)
( ) Em um dicionário de dicionários, uma chave pode apontar para outro dicionário.
( ) `cadastro["A101"]` retorna diretamente a média do aluno.
( ) `cadastro["A101"]["media"]` acessa um campo dentro do sub-dicionário.
( ) Dicionário de dicionários é útil quando cada item possui vários atributos.

Gabarito
(V) Em um dicionário de dicionários, uma chave pode apontar para outro dicionário.
(F) `cadastro["A101"]` retorna o sub-dicionário, não a média.
(V) `cadastro["A101"]["media"]` acessa um campo dentro do sub-dicionário.
(V) Dicionário de dicionários é útil quando cada item possui vários atributos.

Módulo 4: Operações - Trabalhando com a estrutura

15. Como consultar dados

Regra de ouro: Sempre pense em DOIS níveis!

1. **Chave externa** → identifica o item principal (ex: código do aluno)
2. **Chave interna** → identifica o atributo (ex: curso, turno, media)

```
# Acessando com get() - forma segura
dados_aluno = cadastro.get("A101", {})
print(dados_aluno.get("curso", "?"))   # Informática
print(dados_aluno.get("turno", "?"))   # Manhã

# Em uma linha (encadeamento)
print(cadastro.get("A102", {}).get("media", 0.0))  # 7.0
print(cadastro.get("A999", {}).get("media", 0.0))  # 0.0 (sem erro!)
```

15.1. Estratégia mental para não se perder

Sempre se pergunte:

1. **Qual é a chave externa?** (o identificador principal)
2. **O valor dessa chave é o quê?** (um sub-dicionário)
3. **Quais são as chaves internas?** (os nomes dos campos)

---

16. Alterando informações

```
# Primeiro, obtém o dicionário interno com get()
dados_aluno = cadastro.get("A102", {})
```

```
# Depois, altera o campo
dados_aluno["media"] = 8.1

print(cadastro.get("A102", {}).get("media", 0.0))  # 8.1
```

Agora o cadastro ficou:

```
{
    'A101': {'curso': 'Informática', 'turno': 'Manhã', 'media': 8.5},
    'A102': {'curso': 'Administração', 'turno': 'Noite', 'media': 8.1},  # Alterado!
    'A103': {'curso': 'Enfermagem', 'turno': 'Tarde', 'media': 9.2}
}
```

---

17. Adicionando novas informações

E se quisermos adicionar a situação do aluno (aprovado/reprovado)? Basta criar uma nova chave no dicionário interno!

```
#  Adicionando uma nova chave no dicionário interno
dados_aluno = cadastro.get("A101", {})
dados_aluno["situacao"] = "Aprovado"

print(cadastro.get("A101", {}))
# Saída: {'curso': 'Informática', 'turno': 'Manhã', 'media': 8.5, 'situacao': 'Aprovado'}
```

---

18. Percorrendo a estrutura

Forma básica (apenas as chaves externas):

```
for codigo in cadastro:
    print(codigo)
```

Saída:

```
A101
A102
A103
```

Acessando os dados internos:

```
for codigo, dados in cadastro.items():
    print("=" * 30)
    print("Aluno:", codigo)
    print("Curso:", dados.get("curso", "?"))
    print("Turno:", dados.get("turno", "?"))
    print("Média:", dados.get("media", 0.0))
```

Saída:

```
==============================
```

```
Aluno: A101
Curso: Informática
Turno: Manhã
Média: 8.5
==============================
Aluno: A102
Curso: Administração
Turno: Noite
Média: 8.1
==============================
Aluno: A103
Curso: Enfermagem
Turno: Tarde
Média: 9.2
```

18.1. Percorrendo com `items()` (forma mais clara)

O método `items()` permite percorrer chave e valor ao mesmo tempo.

```python
for codigo, dados in cadastro.items():
    print(codigo, dados)
```

Saída:

```text
A101 {'curso': 'Informática', 'turno': 'Manhã', 'media': 8.5}
A102 {'curso': 'Administração', 'turno': 'Noite', 'media': 7.0}
A103 {'curso': 'Enfermagem', 'turno': 'Tarde', 'media': 9.2}
```

Exemplo mais completo:

```python
for codigo, dados in cadastro.items():
    print("=" * 30)
    print("Aluno:", codigo)
    print("Curso:", dados["curso"])
    print("Turno:", dados["turno"])
    print("Média:", dados["media"])
```

⚠ ERRO COMUM:

```python
# ERRADO! Isso vai confundir
for dados in cadastro:
    print(dados["curso"])  # Vai dar erro porque "dados" é a chave, não o valor!
```

💡 IDEIA-CHAVE:

Use `for codigo, dados in cadastro.items():` para ter acesso claro à chave e ao valor.

---

19. Filtros (selecionando dados específicos)

Exemplo 1: Alunos com média maior que 8

```python
print("Alunos com média > 8:")
for codigo, dados in cadastro.items():
    media = dados.get("media", 0.0)
    if media > 8:
        print(f"  {codigo}: {dados['curso']} - {dados['media']}")
```

Saída:

```
Alunos com média > 8:
  A101: Informática - 8.5
  A103: Enfermagem - 9.2
```

Exemplo 2: Alunos do turno da manhã

```python
print("Alunos do turno Manhã:")
for codigo, dados in cadastro.items():
    turno = dados.get("turno","")
    if turno == "Manhã":
        print(f"  {codigo}: {dados['curso']}")
```

Saída:

```
Alunos do turno Manhã:
  A101: Informática
```

Mini Teste 5

1. O que faz `items()`?

a) retorna só as chaves
b) retorna só os valores
c) retorna chave e valor juntos
d) apaga o dicionário

2. Em `dados["media"]`, o que é `"media"`?

a) chave externa
b) chave interna
c) valor externo
d) índice numérico

3. Qual a forma correta de percorrer um dicionário de dicionários?

a) `for codigo in cadastro:`
b) `for codigo, dados in cadastro:`
c) `for codigo, dados in cadastro.items():`
d) `for dados in cadastro.items():`

Gabarito

1. **c)** retorna chave e valor juntos
2. **b)** chave interna
3. **c)** `for codigo, dados in cadastro.items():`

Módulo 5: Aprofundamento - Indo além

20. Agrupamento de dados

Problema: Queremos agrupar os alunos por turno. Ou seja, queremos um novo dicionário onde cada turno tenha uma lista de alunos.

Solução 1: Usando `if chave not in dicionario`

```python
grupo_turno = {}

for codigo, dados in cadastro.items():
    turno = dados["turno"]

    # Verifica se a chave "turno" ainda não existe no dicionário
    if turno not in grupo_turno:
        # Se não existe, cria uma lista vazia
        grupo_turno[turno] = []

    # Adiciona o código do aluno à lista do turno
    grupo_turno[turno].append(codigo)

print(grupo_turno)
```

Saída:

```python
{
    "Manhã": ["A101"],
    "Noite": ["A102"],
    "Tarde": ["A103"]
}
```

20.1. Visualizando o agrupamento

```text
ANTES (dados individuais):
A101 → Manhã
A102 → Noite
A103 → Tarde

DEPOIS (agrupados em um novo dicionário):
Manhã → [A101]
Noite → [A102]
Tarde → [A103]
```

---

21. Agrupamento com `setdefault()` (forma mais curta)

O método `setdefault()` faz exatamente o que fizemos com `if`, mas em uma linha:

```python
grupo_turno = {}

for codigo, dados in cadastro.items():
    turno = dados["turno"]
    # setdefault tenta pegar o valor da chave; se não existir, cria com o valor padrão []
    grupo_turno.setdefault(turno, []).append(codigo)
```

```
print(grupo_turno)  # Mesmo resultado
```

Como funciona `setdefault()`:

- `setdefault(chave, valor_padrao)` tenta obter o valor associado à chave
- Se a chave **não existe**, ela é criada com o `valor_padrao`
- Se a chave **já existe**, retorna o valor existente
- Depois, podemos usar o resultado (no caso, uma lista) e chamar `.append()`

---

22. Contagem de frequências

Problema: Quantos alunos existem em cada turno?

```
freq_turno = {}

for codigo, dados in cadastro.items():
    turno = dados["turno"]
    # get(turno, 0) tenta pegar o valor; se não existir, usa 0
    freq_turno[turno] = freq_turno.get(turno, 0) + 1

print(freq_turno)  # Mesmo resultado
```

Como funciona `get()`:

- `get(chave, valor_padrao)` tenta obter o valor associado à chave
- Se a chave **existe**, retorna o valor
- Se a chave **não existe**, retorna o `valor_padrao` (sem modificar o dicionário)
- No exemplo acima, `freq_turno.get(turno, 0)` retorna o valor atual do turno ou 0

---

23. Comparação entre as abordagens

Objetivo	Usando `if`	Usando método específico
Agrupar elementos em listas	`if chave not in dict: dict[chave] = []` `dict[chave].append(valor)`	`dict.setdefault(chave, []).append(valor)`
Contar frequências	`if chave not in dict: dict[chave] = 0` `dict[chave] += 1`	`dict[chave] = dict.get(chave, 0) + 1`

Vantagens dos métodos `get()` e `setdefault()`:

- Código mais curto e legível
- Menos linhas, menos chance de erro
- Uma vez acostumado, fica mais fácil de ler

---

24. Exercício guiado 3: Pessoas

Vamos criar um exemplo completo do zero, comparando as duas abordagens.

```python
# Dicionários de origem
cidades = {
    "CPF1": "Belo Horizonte",
    "CPF2": "Vitória",
    "CPF3": "Curitiba"
}

idades = {
    "CPF1": 19,
    "CPF2": 21,
    "CPF3": 20
}

profissoes = {
    "CPF1": "Estudante",
    "CPF2": "Técnica",
    "CPF3": "Monitora"
}

# Etapa 1: Primeiro, com tupla (só para comparar)
pessoas_tupla = {}
for cpf in cidades:
    pessoas_tupla[cpf] = (cidades[cpf], idades[cpf], profissoes[cpf])

print("Com tupla:")
print(pessoas_tupla["CPF1"])
# Saída: ('Belo Horizonte', 19, 'Estudante')
# Problema: o que é posição 0, 1 e 2?

# Etapa 2: Agora, com dicionário interno (melhor!)
pessoas = {}
for cpf in cidades:
    pessoas[cpf] = {
        "cidade": cidades[cpf],
        "idade": idades[cpf],
        "profissao": profissoes[cpf]
    }

print("\nCom dicionário interno:")
print(pessoas["CPF1"])
# Saída: {'cidade': 'Belo Horizonte', 'idade': 19, 'profissao': 'Estudante'}

# Agora fica claro o que cada campo significa!
print(pessoas["CPF1"]["cidade"])    # Belo Horizonte
print(pessoas["CPF1"]["idade"])     # 19
print(pessoas["CPF1"]["profissao"]) # Estudante

# Etapa 3: Agrupando por profissão - duas formas

# Forma 1: usando if
```

```python
print("\n--- Agrupamento com if ---")
grupo_profissao_if = {}
for cpf, dados in pessoas.items():
    profissao = dados["profissao"]
    if profissao not in grupo_profissao_if:
        grupo_profissao_if[profissao] = []
    grupo_profissao_if[profissao].append(cpf)
print(grupo_profissao_if)

# Forma 2: usando setdefault (mais curta)
print("\n--- Agrupamento com setdefault ---")
grupo_profissao_sd = {}
for cpf, dados in pessoas.items():
    profissao = dados["profissao"]
    grupo_profissao_sd.setdefault(profissao, []).append(cpf)
print(grupo_profissao_sd)

# Etapa 4: Contando por profissão - duas formas

# Forma 1: usando if
print("\n--- Contagem com if ---")
contagem_if = {}
for cpf, dados in pessoas.items():
    profissao = dados["profissao"]
    if profissao not in contagem_if:
        contagem_if[profissao] = 0
    contagem_if[profissao] += 1
print(contagem_if)

# Forma 2: usando get (mais curta)
print("\n--- Contagem com get ---")
contagem_get = {}
for cpf, dados in pessoas.items():
    profissao = dados["profissao"]
    contagem_get[profissao] = contagem_get.get(profissao, 0) + 1
print(contagem_get)
```

Mini Teste 6

1. Qual estrutura usamos para agrupar elementos por categoria?

a) lista simples
b) dicionário com listas
c) tupla
d) string

2. O que faz `setdefault(turno, [])`?

a) cria um novo dicionário
b) retorna o valor do turno ou cria uma lista vazia se não existir
c) remove o turno do dicionário
d) conta quantos turnos existem

3. O que faz `get(turno, 0)`?

a) retorna o valor do turno ou 0 se não existir
b) cria um novo turno com valor 0
c) remove o turno
d) soma 1 ao valor do turno

4. Qual das opções abaixo é equivalente a este código?

```python
if chave not in dicionario:
    dicionario[chave] = 0
dicionario[chave] += 1
```

a) `dicionario[chave] = dicionario.get(chave, 0) + 1`
b) `dicionario.setdefault(chave, 0) + 1`
c) `dicionario[chave] = dicionario[chave] + 1`
d) `dicionario[chave] += dicionario.get(chave, 0)`

Gabarito

1. **b)** dicionário com listas
2. **b)** retorna o valor do turno ou cria uma lista vazia se não existir
3. **a)** retorna o valor do turno ou 0 se não existir
4. **a)** `dicionario[chave] = dicionario.get(chave, 0) + 1`

Resumo Final e Desafio

25. Resumo: A ideia principal

Quando temos várias informações sobre o mesmo elemento:

- Podemos juntar tudo em uma tupla
- **Mas** se cada posição representa um dado diferente, o ideal é usar **dicionário de dicionários**

Estrutura geral:

```
novo_dicionario = {}

for chave, valor_principal in dicionario_principal.items():
    novo_dicionario[chave] = {
        "campo1": valor_principal,
        "campo2": dicionario2.get(chave, "padrão2"),
        "campo3": dicionario3.get(chave, "padrão3")
    }
```

Vantagem principal:
O código passa a dizer claramente o nome de cada informação. Não precisa decorar posições!

26. Modelo de referência

```python
# Dicionários de origem
d1 = {"ch1": valor1, "ch2": valor2}
d2 = {"ch1": valor3, "ch2": valor4}
d3 = {"ch1": valor5, "ch2": valor6}

# Construindo o dicionário de dicionários
novo = {}

for chave in d1:
    novo[chave] = {
        "campo_a": d1[chave],
        "campo_b": d2[chave],
        "campo_c": d3[chave]
    }
```

---

27. Regras de ouro para lembrar

1. **Se tem NOME → use dicionário**
2. **Se tem POSIÇÃO → use lista/tupla**
3. **Sempre pergunte:** "O que cada valor representa?"
4. **Use** `items()` para percorrer chave e valor juntos
5. **Pense em 2 níveis:** chave externa (quem) e chave interna (o quê)
6. **Use** `get()` **para contagens:** `dict[chave] = dict.get(chave, 0) + 1`
7. **Use** `setdefault()` **para agrupamentos:** `dict.setdefault(chave, []).append(valor)`

---

28. Desafio Final

Enunciado: Monte um dicionário de dicionários chamado `jogos` a partir dos dicionários abaixo:

```python
plataformas = {
    "J1": "PC",
    "J2": "Console",
    "J3": "Mobile"
}

precos = {
    "J1": 89.9,
    "J2": 299.9,
    "J3": 0.0
}

classificacoes = {
    "J1": "14 anos",
    "J2": "Livre",
    "J3": "10 anos"
```

```
}
```

Tarefas:

1. Construa o dicionário `jogos` seguindo o padrão que aprendemos
2. Mostre a plataforma do jogo J2
3. Mostre o preço do jogo J1
4. Mostre a classificação do jogo J3
5. Agrupe os jogos por plataforma (crie um dicionário onde cada plataforma tem uma lista de códigos de jogos) - use **duas formas**: com `if` e com `setdefault()`
6. Conte quantos jogos existem em cada plataforma - use **duas formas**: com `if` e com `get()`

Gabarito

python

```python
# Dicionários de origem
plataformas = {
    "J1": "PC",
    "J2": "Console",
    "J3": "Mobile"
}

precos = {
    "J1": 89.9,
    "J2": 299.9,
    "J3": 0.0
}

classificacoes = {
    "J1": "14 anos",
    "J2": "Livre",
    "J3": "10 anos"
}

# 1. Construindo o dicionário de dicionários
jogos = {}

for codigo in plataformas:
    jogos[codigo] = {
        "plataforma": plataformas[codigo],
        "preco": precos[codigo],
        "classificacao": classificacoes[codigo]
    }

# 2. Mostrando os dados solicitados
print("Plataforma do J2:", jogos["J2"]["plataforma"])
print("Preço do J1:", jogos["J1"]["preco"])
print("Classificação do J3:", jogos["J3"]["classificacao"])

# 3. Agrupamento por plataforma - duas formas

# Forma 1: usando if
print("\n--- Agrupamento com if ---")
grupo_plataforma_if = {}
for codigo, dados in jogos.items():
    plataforma = dados["plataforma"]
    if plataforma not in grupo_plataforma_if:
        grupo_plataforma_if[plataforma] = []
    grupo_plataforma_if[plataforma].append(codigo)
print(grupo_plataforma_if)

# Forma 2: usando setdefault
print("\n--- Agrupamento com setdefault ---")
grupo_plataforma_sd = {}
```

```python
for codigo, dados in jogos.items():
    plataforma = dados["plataforma"]
    grupo_plataforma_sd.setdefault(plataforma, []).append(codigo)
print(grupo_plataforma_sd)

# 4. Contagem por plataforma - duas formas

# Forma 1: usando if
print("\n--- Contagem com if ---")
contagem_if = {}
for codigo, dados in jogos.items():
    plataforma = dados["plataforma"]
    if plataforma not in contagem_if:
        contagem_if[plataforma] = 0
    contagem_if[plataforma] += 1
print(contagem_if)

# Forma 2: usando get
print("\n--- Contagem com get ---")
contagem_get = {}
for codigo, dados in jogos.items():
    plataforma = dados["plataforma"]
    contagem_get[plataforma] = contagem_get.get(plataforma, 0) + 1
print(contagem_get)

# Bônus: Mostrar todos os jogos
print("\n--- Todos os jogos ---")
for codigo, dados in jogos.items():
    print(f"\nJogo {codigo}:")
    print(f"  Plataforma: {dados['plataforma']}")
    print(f"  Preço: R$ {dados['preco']}")
    print(f"  Classificação: {dados['classificacao']}")
```

Saída esperada:

```text
Plataforma do J2: Console
Preço do J1: 89.9
Classificação do J3: 10 anos

--- Agrupamento com if ---
{'PC': ['J1'], 'Console': ['J2'], 'Mobile': ['J3']}

--- Agrupamento com setdefault ---
{'PC': ['J1'], 'Console': ['J2'], 'Mobile': ['J3']}

--- Contagem com if ---
{'PC': 1, 'Console': 1, 'Mobile': 1}

--- Contagem com get ---
{'PC': 1, 'Console': 1, 'Mobile': 1}

--- Todos os jogos ---

Jogo J1:
  Plataforma: PC
  Preço: R$ 89.9
  Classificação: 14 anos

Jogo J2:
  Plataforma: Console
  Preço: R$ 299.9
  Classificação: Livre

Jogo J3:
```

```
Plataforma: Mobile
Preço: R$ 0.0
Classificação: 10 anos
```

---

29. Encerramento

Parabéns! Você aprendeu um dos conceitos mais importantes em programação com Python.

O que você conquistou:

- ✅ Entendeu a diferença entre tuplas e dicionários internos
- ✅ Aprendeu a construir dicionários de dicionários
- ✅ Sabe consultar, alterar e percorrer essa estrutura
- ✅ Consegue filtrar, agrupar e contar dados
- ✅ Aprendeu a usar `get()` para contagens
- ✅ Aprendeu a usar `setdefault()` para agrupamentos
- ✅ Identifica quando usar cada estrutura

Onde usar isso na vida real:

- Sistemas acadêmicos (alunos, cursos, notas)
- Cadastros de clientes
- Bancos de dados simples
- Análise de dados
- Relatórios

Lembre-se sempre:

Código bom é aquele que outra pessoa entende SEM explicação. Use nomes claros para seus campos e sua estrutura explicará o que faz.

Glossário Rápido

Termo	Significado
Dicionário	Estrutura que guarda dados em pares chave → valor
Chave	Identificador único que acessa um valor
Valor	Informação associada à chave
Tupla	Estrutura que guarda dados em posições (0, 1, 2...)

Termo	Significado
Dicionário de dicionários	Dicionário onde cada valor é outro dicionário
items()	Método que retorna pares (chave, valor)
get()	Método que retorna o valor da chave ou um valor padrão se a chave não existir
setdefault()	Método que retorna o valor da chave; se não existir, insere a chave com o valor padrão

---

Material produzido com dedicação para alunos que aprendem no seu próprio ritmo.

Pratique, erre, corrija, aprenda. Cada erro é um passo em direção ao acerto!